UNIVERSAL CODE SCANNER

18 · Guía para nuevos desarrolladores

Documentación de sistema

Versión analizada: **1.1.0+2**
Commit: **6d5e0b7**
Fecha del documento: **2026-08-27**

Documento generado desde el Markdown de **docs/system-documentation/** con **tool/build_system_documentation_pdf.py**. El Markdown es la fuente: no editar este PDF.

Contenido

1. Antes de nada: la sorpresa número uno
2. Día 1 · Entorno y primer arranque
3. Día 1 · Qué leer, en este orden
4. Día 2 · El modelo mental
5. Día 2 · Convenciones que hay que seguir
6. Lo que NO puedes tocar sin leer antes
7. Cómo hacer los cambios más habituales
8. Primeras tareas sugeridas
9. Antes de cada commit
10. Dónde buscar cuando algo no cuadra
11. Diez cosas que descubrirías a golpes

Itinerario de incorporación. Está pensado para que alguien que llega hoy pueda hacer un cambio útil y seguro **al final del segundo día**.

Antes de nada: la sorpresa número uno

Un clon recién hecho no compila. No es un fallo: `.gitignore` excluye `/android/`, `/ios/`, `/web/` y `/macos/` con el comentario «Se generan con tool/bootstrap.py».

```
python tool/bootstrap.py --platforms android,web
```

Si te saltas ese paso perderás media hora buscando qué falta. Todo lo demás de esta guía asume que ya lo ejecutaste.

Día 1 · Entorno y primer arranque

Instalar

Requisito	Versión	Cómo comprobarlo
Flutter	**3.44.7**	`flutter --version`
Dart	Viene con Flutter	`dart --version`
Python	3.x	`python --version`
`pyyaml`	Cualquiera reciente	`python -c "import yaml"`

La versión de Flutter está fijada en `../../.fvmrc` y replicada en la variable `FLUTTER_VERSION` de ambos workflows. **Nada la comprueba automáticamente**: si usas otra, `pub get` podría resolver distinto y `bootstrap.py` generar plantillas distintas.

Los cinco comandos que vas a repetir

```
flutter pub get                            # resolver dependencias
python tool/bootstrap.py --platforms android,web
python tool/validate_structure.py          # diez comprobaciones offline, < 1 s
flutter analyze --fatal-infos              # el nivel que usa la CI
flutter test                               # 57 casos, ~24 s
```

Memoriza el tercero: es el gate propio del repositorio, corre sin toolchain y te avisa de enlaces rotos, versiones descuadradas y rutas de tu máquina antes de que lleguen a la CI.

Aviso: `tool/run_quality_gate.sh` ejecuta `flutter analyze` **sin** `--fatal-infos`. Puede pasar en verde y fallar en la CI. Usa siempre la forma estricta a mano. Es el hallazgo R-06.

Arrancar

```
flutter run              # dispositivo o emulador
flutter run -d chrome    # PWA en local, sin --base-href
```

Día 1 · Qué leer, en este orden

No leas 6 821 líneas de golpe. Este recorrido cubre las decisiones importantes en unas dos horas.

#	Archivo	Líneas	Por qué
1	`lib/main.dart`	14	Tres decisiones en tres líneas
2	`lib/bootstrap.dart`	77	Todo el grafo de servicios, en orden
3	`lib/models/scan_record.dart`	161	El modelo central; `fromBarcode` es la confluencia de todo
4	`lib/core/security/payload_cipher.dart`	168	La pieza más delicada del sistema
5	`lib/core/security/scan_security_analyzer.dart`	165	Las 16 señales, todas en una función
6	`lib/features/scanner/widgets/scan_status_bar.dart`	137	Ejemplo del estilo de comentarios del proyecto
7	`lib/services/history_repository.dart`	223	Persistencia, migración y recorte

Y después, dos documentos del repositorio que valen más que cualquier resumen:

- `../quality/SCANNER_UX.md` — explica un defecto real y por qué se resolvió así. Es la mejor pieza de documentación del proyecto.
- `../quality/COMPATIBILITY_CONTRACT.md` — los 14 compromisos que **no puedes romper**.

Truco para leer un archivo grande

Extrae primero el mapa de símbolos:

```
grep -n "^\s*\(class \|abstract \|Future<\|static \|void \|String \|bool \)" \
 lib/features/scanner/scanner_screen.dart
```

Después lee en profundidad solo lo que importe.

Día 2 · El modelo mental

Cinco capas, dependencia hacia abajo

```
features/  pantallas por caso de uso
  ↓
state/     ChangeNotifier: ScanStore, InventoryStore, SettingsStore
  ↓
services/  interpretación, importación, exportación, repositorios
  ↓
models/    entidades inmutables y serializables
  ↓
core/      base de datos, cifrado, recuperación, tema, i18n, diagnóstico
```

Dos excepciones reales que verás en cuanto abras el código, y que están registradas como hallazgos: `models/scan_record.dart` importa `features/formats/domain/` (R-07), y `core/recovery/recovery_service.dart` importa `models/` para validar cargas descifradas.

El flujo que hay que tener en la cabeza

```
Cámara / Imagen / PDF
       ↓
ScanRecord.fromBarcode
       ■■■ ScanSecurityAnalyzer.analyze  → nivel de riesgo, canOpen
       ■■■ ContentParserRegistry.parse   → tipo, campos, sensible
       ↓
ScanResultsSheet  ← siempre se muestra todo
       ↓
¿saveHistory && !privateMode && !isSensitive?
       ↓ sí
PayloadCipher.encryptJson → Sembast
```

Interioriza esto: interpretación y análisis ocurren **antes** de que exista la posibilidad de acción, y la persistencia es una rama independiente de la acción.

Sin gestor de estado, sin router, sin DI

No hay Riverpod, Bloc, Provider, GetIt ni GoRouter. Los servicios se construyen una vez en `AppBootstrapper.initialize` y se pasan **por constructor** hasta las pantallas. La navegación entre pestañas es un índice entero sobre una lista de cinco widgets.

Puede parecer poco sofisticado. Es deliberado y funciona para el tamaño del proyecto: no hay indirección que rastrear.

Día 2 · Convenciones que hay que seguir

Estilo de Dart

`analysis_options.yaml` añade cuatro reglas a `flutter_lints`:

Regla	Qué te obliga a hacer
`always_declare_return_types`	Declarar el tipo de retorno siempre
`avoid_print`	**No usar `print`**; es control de privacidad, no de estilo
`prefer_final_locals`	`final` en toda variable local que no se reasigne
`use_build_context_synchronously`	`if (!mounted) return;` tras cada `await` antes de tocar el cont exto

La última es la que más aparece. El patrón del repositorio es:

```
await algoAsincrono();
if (!mounted) return;
setState(() { … });
```

y, dentro de un callback con `BuildContext` propio:

```
if (context.mounted) ScaffoldMessenger.of(context).showSnackBar(…);
```

Tipos explícitos

El proyecto escribe los tipos completos incluso donde `var` bastaría:

```
final List<RecordSnapshot<String, Map<String, Object?>>> snapshots = await _store.find(…);
```

Es verboso a propósito y hace el código buscable con `grep`. Síguelo.

Comentarios: el porqué, no el qué

Esta es la convención más importante. El repositorio tiene **130 líneas de comentario en 6 821 de código** —densidad baja— pero las que hay son de calidad alta. Todas explican una decisión.

Buen ejemplo real, de `scanner_screen.dart`:

```
// The old controller is released before the new one exists: both share a
// single platform camera session, and disposing it afterwards would tear
// down the session the new controller has just claimed.
```

Contraejemplo de lo que **no** hay que escribir: `// asigna el valor`.

Los comentarios existentes están en inglés; el texto de la interfaz, en español. Sigue esa separación.

Textos de interfaz

Solo doce cadenas viven en `AppLocalizations` —las cinco de navegación, título y subtítulo del escáner y algunas más—. El resto son literales en español dentro de cada pantalla.

Si añades una cadena nueva a la interfaz, añádela a `AppLocalizations`, no como literal. Es lo que exige `docs/quality/ACCESSIBILITY.md` y es el requisito para poder exponer el inglés algún día.

Serialización

Toda entidad persistida implementa `toJson()` y `fromJson()` a mano. **No hay generación de código**: ni `json_serializable`, ni `freezed`, ni `build_runner`.

Las enumeraciones se serializan **por nombre** (`.name` / `.byName`). Renombrar un valor de `ContentKind`, `RiskLevel`, `RecoveryEntityType` o `RecoveryIssueState` **rompe los datos ya guardados y los respaldos exportados**.

Pruebas

Los dobles se escriben a mano implementando la interfaz. No hay `mockito` ni `mocktail`. Ejemplos a copiar: `_FakeTonePlayer`, `_TrackingKeyProvider`, `_TestParser`.

Patrones obligatorios:

```
// Base en memoria
final AppDatabase database = await AppDatabase.openTemporary();
addTearDown(database.close);

// Llaves en memoria
PayloadCipher(keyProvider: MemoryEncryptionKeyProvider());

// Preferencias en memoria
SharedPreferencesAsyncPlatform.instance = InMemorySharedPreferencesAsync.empty();
```

Y dos prohibiciones aprendidas a base de defectos reales:

1. **Nunca `pumpAndSettle` sobre una animación infinita.** La barra indeterminada del escáner nunca para: `pumpAndSettle` se cuelga. Usa `await tester.pump()`.
2. **Nunca abras la base dentro del reloj falso de `testWidgets`.** Sembast usa temporizadores reales. Envuelve en `tester.runAsync(...)`.

Ambas están comentadas en el propio código de las pruebas.

Finales de línea

`.gitattributes` fuerza `* text=auto eol=lf` porque **los SHA-256 de `SOURCE_MANIFEST.json` dependen de ello**. Cualquier archivo de texto nuevo debe escribirse con LF. La única excepción es `*.ps1`, que va con CRLF para que PowerShell lo lea de forma nativa.

Lo que NO puedes tocar sin leer antes

Seis constantes cuya modificación **convierte los datos del usuario en ruido**, sin reversión posible:

Constante	Archivo
`PayloadCipher.currentVersion`	`lib/core/security/payload_cipher.dart`
`PayloadCipher.currentAlgorithm`	idem
`PayloadCipher.currentKeyId`	idem
`PayloadCipher.legacyKeyId`	idem
`SecureStorageKeyProvider._prefix`	idem
`SchemaMigrator.currentVersion`	`lib/core/database/schema_migrator.dart`

No es reversible porque la **actualización perezosa de sobres** reescribe los registros antiguos la primera vez que se leen. Antes de tocar cualquiera de las seis, lee 07-database.md y `../quality/COMPATIBILITY_CONTRACT.md`.

Y tres versiones de dependencia que **rompen `pub get`** si las subes:

Paquete	Tope	Motivo
`share_plus`	`12.0.2`	Desde `13.0.0` exige `win32 ^6`, incompatible con `file_picker 11.x`
`pdfrx`	`2.4.5`	Desde `2.4.6` arrastra `archive ^4`, incompatible con `excel 4.0.6`
`excel`	`4.0.6`	Es la que ancla `archive` a la serie 3

`flutter pub outdated` te dirá que se pueden subir. **No los subas.** El motivo está en `../quality/LOCKFILE.md` y replicado en `.github/dependabot.yml`.

Cómo hacer los cambios más habituales

Añadir un intérprete de contenido nuevo

El camino limpio, sin tocar el intérprete integrado:

```
class MiParser implements ContentParser {
 @override String get id => 'mi-parser';
 @override int get priority => 100;          // el integrado tiene -1000
 @override bool canParse(String raw) => raw.startsWith('MIFORMATO:');
 @override ParsedContent parse(String raw) => ParsedContent(
```

```
      kind: ContentKind.text,
      title: 'Mi formato',
      fields: <String, String>{ 'Campo': raw },
      sensitive: false,
    );
}

ContentParserRegistry.instance.register(MiParser());
```

Cuidado con `sensitive`: decide si el registro entra o no en el historial. Y si tu parser produce un campo con datos delicados, **debe llamarse exactamente `Contraseña`, `Secreto`, `Dirección`, `IBAN` o `Carga`** para que se enmascare. Es una lista literal en `scan_result_sheet.dart`, hallazgo R-19.

No hay ningún punto en la aplicación desde el que se registre un parser: la extensibilidad es en tiempo de compilación, para quien bifurque el repositorio.

Añadir un ajuste

Cinco archivos, en este orden:

1. `lib/models/app_settings.dart` — campo, valor por defecto y `copyWith`;
2. `lib/services/settings_repository.dart` — leerlo en `load` y escribirlo en `save`;
3. decidir si va en la lista de `resetNonSensitive()` —los de seguridad **no** van: `save_history`, `private_mode` y `biometric_lock` quedan fuera a propósito;
4. `lib/features/settings/settings_screen.dart` — el control;
5. donde se consuma.

Añadir una señal de riesgo

Una sola línea en `ScanSecurityAnalyzer.analyze`:

```
if (miCondicion(uri)) caution('Texto en español que explica qué se detectó.');
```

`caution` solo eleva desde `low`; `high` es absorbente. **Añade también la prueba en `test/scan_security_analyzer_test.dart`** —es la única defensa antes de una acción externa— y actualiza el número «16» en `README.md`, `CHANGELOG.md`, `IMPLEMENTATION_STATUS.md` y en 05-technical-reference.md.

Añadir un campo persistido

1. Añádelo al modelo, a `toJson` y a `fromJson` **con valor por defecto** en la lectura: es lo que hace que un registro antiguo siga cargando.
2. **No subas `PayloadCipher.currentVersion`**: el sobre no cambia, cambia su contenido.
3. Si el campo es obligatorio y no puede tener valor por defecto, hace falta una migración explícita.

Subir la versión

Dos archivos, siempre juntos:

1. `pubspec.yaml`, campo `version`, forma `X.Y.Z+N`;
2. `lib/core/app_info.dart`, constante `appVersion`, parte `X.Y.Z`.

`validate_structure.py` **falla si divergen**. Ese gate nació de un defecto real: la pantalla «Acerca de» mostró `1.0.0` durante toda la versión `1.1.0`.

Primeras tareas sugeridas

Ordenadas por dificultad creciente. Las cuatro primeras son cambios reales y útiles que no tocan nada delicado.

#	Tarea	Dificultad	Qué te enseña
1	Añadir `--fatal-infos` a `tool/run_quality_gate.sh` (R-06)	Trivial	El sistema de gates
2	Añadir la columna `notas` al CSV del historial (R-16)	Baja	`ExportService` y el isolate de `compute`
3	Escribir pruebas para `_escape` y `_normalizeUrl` del generador	Baja	Las convenciones de prueba
4	Eliminar `BarcodeLabels.contentType` y `_fallbackType` (R-05)	Baja	Verificar que algo es código muerto antes de borrarlo
5	Escribir la prueba de que un registro sensible **no** entra en el historial	Media	El filtro de privacidad y los dobles
6	Probar `_trim` y `pruneOlderThan`	Media	Sembast y los `Finder`
7	Probar la migración del historial desde preferencias (R-34)	**Alta**	El flujo más delicado del proyecto
8	Resolver R-03: filtrar contenido sensible en el inventario	**Alta**	Requiere decisión de producto, no solo código

Antes de la 7 y la 8, habla con quien mantenga el proyecto: la 7 toca el único flujo que puede perder datos, y la 8 cambia comportamiento observable.

Antes de cada commit

```
python tool/validate_structure.py
flutter analyze --fatal-infos
flutter test
```

Si tocaste archivos versionados nuevos, además:

```
python tool/generate_source_manifest.py
```

Y si tocaste la documentación de `docs/system-documentation/`:

```
python tool/build_system_documentation_pdf.py
```

Dónde buscar cuando algo no cuadra

Pregunta	Documento

¿Qué hace esta función?	05 · Referencia técnica
¿Por qué está escrita así?	06 · Explicación profunda
¿Dónde vive esto?	04 · Mapa del código
¿Cómo se guarda?	07 · Base de datos
¿Por dónde entra y sale este dato?	08 · Flujo de datos
¿Qué ajuste controla esto?	10 · Configuración
Me sale este error	14 · Solución de problemas
¿Esto es un defecto conocido?	15 · Riesgos y deuda técnica
¿Qué significa esta palabra?	16 · Glosario
¿Dónde se prueba esta funcionalidad?	19 · Matriz de trazabilidad

---

Diez cosas que descubrirías a golpes

Aquí están, para que no tengas que descubrirlas así:

1. **Un clon no compila** sin `python tool/bootstrap.py`.
2. **Leer el historial escribe en la base**, si algún sobre está desactualizado.
3. **El momento de cada lectura queda en claro** en la base; el contenido no.
4. **Un registro que no descifra no rompe la carga**: se omite y se registra como incidencia.
5. **El inventario sí guarda contenido sensible**; el escáner no.
6. **`autoTorch` no se aplica en el inventario**, solo en la pestaña Escanear.
7. **El temporizador del portapapeles sobrevive** al widget que lo creó.
8. **Los diagnósticos se pierden al cerrar**: viven solo en memoria.
9. **Las llaves antiguas no se borran** tras una rotación exitosa.
10. **Restablecer preferencias visuales no toca** `save_history`, `private_mode` ni `biometric_lock`.

Continúa en 19 · Matriz de trazabilidad.